

SKILL WEEK – 4 OSSP

TASK 1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <ctype.h>
```

```
#define MAX_INPUT 1024
```

```
#define MAX_ARGS 100
```

```
/*
```

```
 * Add one character to the current argument.
```

```
*/
```

```
void add_char(char *arg, int *pos, char c)
```

```
{
```

```
    if (*pos < MAX_INPUT - 1)
```

```
    {
```

```
        arg[*pos] = c;
```

```
        (*pos)++;
```

```
        arg[*pos] = '\0';
```

```
    }
```

```
}
```

```
/*
```

```
 * Expand environment variables such as $USER.
```

```
*/
```

```

void expand_variable(const char *input, int *i, char *arg, int *pos)
{
    char variable[100];
    int vpos = 0;

    (*i)++; // Skip $

    while (input[*i] != '\0' &&
           (isalnum((unsigned char)input[*i]) ||
            input[*i] == '_'))
    {
        if (vpos < 99)
        {
            variable[vpos++] = input[*i];
        }
        (*i)++;
    }

    variable[vpos] = '\0';

    const char *value = getenv(variable);

    if (value != NULL)
    {
        for (int j = 0; value[j] != '\0'; j++)
        {
            add_char(arg, pos, value[j]);
        }
    }
}

```

```
    (*i)--; // Adjust because main loop increments i  
}
```

```
/*  
 * Tokenize input.  
 *  
 * Supports:  
 * - Spaces  
 * - Single quotes  
 * - Double quotes  
 * - Escape sequences  
 * - Environment variables inside double quotes  
 */
```

```
int tokenize(char *input, char **args)  
{  
    int argc = 0;  
    int i = 0;  
  
    while (input[i] != '\0')  
    {  
        // Skip whitespace  
        while (isspace((unsigned char)input[i]))  
        {  
            i++;  
        }  
  
        if (input[i] == '\0')  
            break;
```

```
char *arg = malloc(MAX_INPUT);
```

```
if (arg == NULL)
```

```
{
```

```
    perror("malloc");
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
int pos = 0;
```

```
arg[0] = '\0';
```

```
while (input[i] != '\0' &&
```

```
       !isspace((unsigned char)input[i]))
```

```
{
```

```
    /*
```

```
     * Single quotes
```

```
    */
```

```
    if (input[i] == '"')
```

```
    {
```

```
        i++;
```

```
        while (input[i] != '\0' && input[i] != '"')
```

```
        {
```

```
            add_char(arg, &pos, input[i]);
```

```
            i++;
```

```
        }
```

```
        if (input[i] != '"')
```

```

{
    printf("Syntax Error: Unmatched single quote.\n");
    free(arg);

    for (int k = 0; k < argc; k++)
        free(args[k]);

    return -1;
}

i++;
}

/*
 * Double quotes
 */
else if (input[i] == '"')
{
    i++;

    while (input[i] != '\0' && input[i] != '"')
    {
        /*
         * Variable expansion inside double quotes
         */
        if (input[i] == '$')
        {
            expand_variable(input, &i, arg, &pos);
            i++;

```

```

    }

    /*
    * Escape sequence inside double quotes
    */
    else if (input[i] == '\\')
    {
        i++;

        if (input[i] != '\0')
        {
            add_char(arg, &pos, input[i]);
            i++;
        }
    }
    else
    {
        add_char(arg, &pos, input[i]);
        i++;
    }
}

if (input[i] != "")
{
    printf("Syntax Error: Unmatched double quote.\n");
    free(arg);

    for (int k = 0; k < argc; k++)
        free(args[k]);
}

```

```

        return -1;
    }

    i++;
}

/*
 * Escape sequence
 */
else if (input[i] == '\\')
{
    i++;

    if (input[i] != '\0')
    {
        add_char(arg, &pos, input[i]);
        i++;
    }
}

/*
 * Normal character
 */
else
{
    add_char(arg, &pos, input[i]);
    i++;
}

```

```

    }

    args[argc++] = arg;

    if (argc >= MAX_ARGS - 1)
        break;
}

args[argc] = NULL;

return argc;
}

/*
 * Execute the command.
 */
void execute_command(char **args)
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

```

```
/*
 * Child process
 */
if (pid == 0)
{
    printf("Child Process PID: %d\n", getpid());

    execvp(args[0], args);

    /*
     * execvp returns only if an error occurs.
     */
    fprintf(stderr, "%s: command not found or execution failed\n",
            args[0]);

    exit(127);
}

/*
 * Parent process
 */
else
{
    printf("Parent Process PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
    }
}
```

```

        return;
    }

    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
}

/*
 * Main shell loop.
 */
int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    MY SIMPLE UNIX SHELL\n");
    printf("=====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

```

```
/*  
 * Read command  
 */  
if (fgets(input, sizeof(input), stdin) == NULL)  
{  
    printf("\n");  
    break;  
}
```

```
/*  
 * Remove newline  
 */  
input[strcspn(input, "\n")] = '\0';
```

```
/*  
 * Handle empty command  
 */  
int empty = 1;
```

```
for (int i = 0; input[i] != '\0'; i++)  
{  
    if (!isspace((unsigned char)input[i]))  
    {  
        empty = 0;  
        break;  
    }  
}
```

```
if (empty)
```

```
{
    continue;
}

/*
 * Tokenization
 */
int argc = tokenize(input, args);

if (argc == -1)
{
    continue;
}

if (argc == 0)
{
    continue;
}

/*
 * Built-in exit command
 */
if (strcmp(args[0], "exit") == 0)
{
    for (int i = 0; i < argc; i++)
        free(args[i]);

    break;
}
```

```
/*
 * Display parsing result
 */
printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

/*
 * Execute command
 */
printf("\n");

execute_command(args);

printf("Command execution completed.\n\n");

/*
 * Free allocated memory
 */
for (int i = 0; i < argc; i++)
{
    free(args[i]);
}
}
```

```
printf("Shell terminated.\n");
```

```
return 0;
```

```
}
```

```
nikith@NIKITH:~$ ls
File1.txt      fork_example.c  process_states  week4_task1
File2.txt      fork_exec       process_states.c week4_task1.c
OSSP          fork_exec.c    shell.c         zombie
command_executor hello          task1.c        zombie.c
command_executor.c hello.c        task3          zombie_fixed
filecopy       my_shell       task3.c        zombie_fixed.c
filecopy.c     myshell        task4
fork_example   myshell.c     task4.c
nikith@NIKITH:~$ ./myshell
=====
MY SIMPLE UNIX SHELL
=====
Type 'exit' to quit.

myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Parent Process PID: 882
Created Child PID: 883
Child Process PID: 883
Hello
Child exited with status: 0
Command execution completed.

myShell> echo Hello World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Argument 3: World

Parent Process PID: 882
Created Child PID: 884
Child Process PID: 884
Hello World
Child exited with status: 0
Command execution completed.
```

TASK 2

```
myShell> echo 'Hello World'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Parent Process PID: 882
Created Child PID: 887
Child Process PID: 887
Hello World
Child exited with status: 0
Command execution completed.

myShell> echo 'This is a test'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: This is a test

Parent Process PID: 882
Created Child PID: 890
Child Process PID: 890
This is a test
Child exited with status: 0
Command execution completed.

myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
```

TASK 3

```
Syntax Error: Unmatched single quote.
myShell> exit
Shell terminated.
nikith@NIKITH:~$ echo $USER
nikith
nikith@NIKITH:~$ ./myshell
=====
MY SIMPLE UNIX SHELL
=====
Type 'exit' to quit.

myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello nikith

Parent Process PID: 913
Created Child PID: 914
Child Process PID: 914
Hello nikith
Child exited with status: 0
Command execution completed.

myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Parent Process PID: 913
Created Child PID: 917
Child Process PID: 917
Hello World
Child exited with status: 0
Command execution completed.
```

TASK 4

```
nikith@NIKITH:~$ ./myshell
=====
MY SIMPLE UNIX SHELL
=====
Type 'exit' to quit.

myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Parent Process PID: 928
Created Child PID: 929
Child Process PID: 929
Hello World
Child exited with status: 0
Command execution completed.

myShell> echo \$USER

--- Parsed Arguments ---
Argument 1: echo
Argument 2: $USER

Parent Process PID: 928
Created Child PID: 932
Child Process PID: 932
$USER
Child exited with status: 0
Command execution completed.
```

TASK 5

--- Parsed Arguments ---

Argument 1: ls

Parent Process PID: 928

Created Child PID: 939

Child Process PID: 939

File1.txt	fork_example.c	process_states	week4_task1
File2.txt	fork_exec	process_states.c	week4_task1.c
OSSP	fork_exec.c	shell.c	zombie
command_executor	hello	task1.c	zombie.c
command_executor.c	hello.c	task3	zombie_fixed
filecopy	my_shell	task3.c	zombie_fixed.c
filecopy.c	myshell	task4	
fork_example	myshell.c	task4.c	

Child exited with status: 0

Command execution completed.

myShell> pwd

--- Parsed Arguments ---

Argument 1: pwd

Parent Process PID: 928

Created Child PID: 940

Child Process PID: 940

/home/nikith

Child exited with status: 0

Command execution completed.

myShell> date

--- Parsed Arguments ---

Argument 1: date

Parent Process PID: 928

Created Child PID: 941

Child Process PID: 941

Mon Aug 24 05:41:13 UTC 2026

Child exited with status: 0

Command execution completed.